AgroManejo

Especificações técnicas para o desenvolvimento

Novembro de 2026

Especificação 01 — Protocolo de sincronização

Complementa a seção 7.4 da proposta técnica (`entrega-basica/AgroManejo-Entrega-Basica.md`). Define o comportamento necessário para implementar a sincronização entre o TinyDB do aparelho e o PostgreSQL (Supabase) sem perda nem duplicação de dados. O aplicativo é feito no MIT App Inventor, apenas com componentes nativos: o acesso ao servidor usa o componente Web, e os disparos usam o componente Clock.

1. Princípios

1. **O aparelho é a fonte de trabalho; o servidor é a fonte de verdade compartilhada.** O produtor lê e grava sempre no TinyDB. O servidor decide se uma alteração é aceita.
2. **O relógio do aparelho nunca decide nada.** Ordem, versão e cursor vêm do servidor.
3. **Toda operação pode ser repetida sem efeito colateral** (idempotência).
4. **Nenhuma alteração é sobrescrita sem que o produtor a tenha visto.**

2. Unidades de sincronização

Unidade	Conteúdo	Observação
Unidade de manejo	Dados da unidade **e** todos os seus vértices	Bloco único: o contorno inteiro é aceito ou recusado junto. Os vértices nunca são sincronizados isoladamente.
Atividade	Todos os campos da atividade	Independente da unidade; referencia a unidade pelo identificador.
Propriedade	Dados da propriedade	**Somente recebimento.** Criada pelo servidor com a conta; o aparelho não a envia nesta versão.

A **exclusão** não é uma operação à parte: é uma alteração que preenche `excluido_em` e segue o mesmo envio, com versão-base e detecção de conflito. Um registro excluído continua no aparelho, oculto, até ser aceito pelo servidor. Ao excluir uma unidade, o aparelho envia só a unidade e apenas oculta as suas atividades: quem as exclui é o servidor, na mesma transação (seção 5), e o recebimento traz essas exclusões de volta. Assim, a exclusão em cascata não gera conflitos com as versões das atividades.

3. Estado de cada registro no aparelho

Situação	Significado	Pode ser editado?
SINCRONIZADO	Igual à versão do servidor.	Sim (passa a PENDENTE)

Situação	Significado	Pode ser editado?
PENDENTE	Alterado no aparelho e ainda não aceito pelo servidor.	Sim
CONFLITO	O servidor tem uma versão mais nova feita em outro aparelho.	**Não**, até o produtor resolver (UC10)
REJEITADO	O servidor recusou por regra de validação. O motivo fica registrado.	Sim (a correção volta a PENDENTE)

Cada registro guarda também:

- **versão-base:** a versão do servidor sobre a qual a edição foi feita (nula em registros novos);
- **identificador da operação pendente:** um UUID gerado quando o registro passa a PENDENTE e regenerado a cada nova edição local. Garante a idempotência.

O bloqueio de edição em CONFLITO evita conflitos sobre conflitos, que o produtor não conseguiria entender.

4. Ciclo de sincronização

Gatilhos (só com o aplicativo aberto, porque os componentes nativos do App Inventor não executam em segundo plano):

- abertura do aplicativo (evento Initialize da tela principal);
- 5 segundos após a última gravação local (Clock de disparo único);
- a cada 60 segundos, enquanto houver registros PENDENTE (Clock periódico);
- fim do login;
- botão “Sincronizar agora”, em Mais → Sincronização.

O App Inventor não tem componente nativo que informe a volta da conexão. A tentativa de envio serve como teste: se falhar por rede, o ciclo termina e a próxima tentativa segue a seção 8. Sem pendências, o Clock periódico fica desligado.

Só um ciclo roda por vez, controlado por uma variável global; um gatilho durante um ciclo agenda o próximo.

Ordem:

1. Renovar a sessão, se o token estiver vencido.
2. **Envio** (seção 5), em lotes de até 100 registros PENDENTE.
3. **Recebimento** (seção 6), em páginas de até 500 alterações, até não haver mais.
4. Gravar o horário do último ciclo bem-sucedido, informado pelo servidor.

O envio vem antes do recebimento para que o conflito seja detectado pelo servidor, com versão, e não por sobrescrita local.

5. Envio de alterações

O aplicativo chama **uma função do banco** (RPC do Supabase), não as tabelas diretamente. No App Inventor, isso é um `PostText` do componente Web para `https://<projeto>.supabase.co/rest/v1/rpc/enviar_alteracoes`, com o lote em JSON (montado com `JsonObjectEncode`) e os cabeçalhos `apikey`, `Authorization: Bearer <token>` e `Content-Type: application/json`, definidos em `RequestHeaders`. A resposta chega no evento `GotText` e é lida com `JsonTextDecodeWithDictionaries`. Uma função do PostgreSQL executa dentro de uma transação, então a gravação da unidade com todos os vértices é atômica.

Conteúdo do lote, para cada registro:

- identificador do registro;
- identificador da operação;
- tipo (unidade ou atividade);
- versão-base;
- dados completos (para a unidade, inclui a lista ordenada de vértices).

O aplicativo também envia a sua **versão**.

Processamento no servidor, registro a registro, cada um isolado dos demais, para que um conflito não bloqueie o lote:

1. Se o identificador da operação já foi processado, devolver o mesmo resultado anterior (idempotência).
2. Conferir que o registro pertence ao produtor autenticado.
3. Validar:
    - polígono válido, reconstruído a partir dos vértices com as funções do PostGIS;
    - no mínimo 3 vértices;
    - campos obrigatórios presentes;
    - valores monetários, previstos e realizados, não negativos;
    - avaliação somente em atividade CONCLUIDA, e CONCLUIDA somente com data realizada;
    - atividade nova ou alterada somente em unidade não excluída.
4. Comparar a versão-base com a versão atual:
    - registro inexistente e versão-base nula → **inserir** com versão 1;
    - versões iguais → **gravar** e incrementar a versão;
    - versões diferentes → **conflito**: nada é gravado.
5. Para unidades: recalcular a área com o PostGIS e gravar esse valor, que passa a ser a área oficial. Se a alteração aceita for uma exclusão, excluir na mesma transação as atividades ainda ativas da unidade, incrementando a versão de cada uma.

6. Registrar a alteração (e, numa exclusão de unidade, cada atividade excluída junto) no registro de alterações (seção 6) e o identificador da operação como processado.

Resposta por registro:

Resultado	Dados devolvidos	Ação no aparelho
ACEITO	nova versão, área oficial (unidades)	SINCRONIZADO; versão-base = nova versão; atualizar a área
CONFLITO	versão atual do servidor	CONFLITO; manter a versão local intacta
REJEITADO	motivo da recusa	REJEITADO; mostrar o motivo ao produtor

Se a resposta se perder (queda de conexão), o reenvio com o mesmo identificador de operação devolve o mesmo resultado, sem gravar de novo.

6. Recebimento de alterações (cursor)

Problema a evitar: um contador simples pode pular registros. Uma transação que começou antes pode terminar depois e ganhar um número menor que um cursor já lido.

Mecanismo (função `receber_alteracoes`, chamada da mesma forma pelo componente Web):

- O servidor mantém um **registro de alterações**: cada gravação aceita gera uma linha com número sequencial, produtor, tipo, identificador do registro, nova versão e o **identificador da transação** que a gravou.
- O recebimento devolve apenas as alterações:
  - do produtor autenticado,
  - com número maior que o cursor do aparelho,
  - **cujas transações já terminaram antes do início de qualquer transação ainda em andamento.** No PostgreSQL 13 ou superior, isso usa as funções `pg_current_xact_id()`, `pg_current_snapshot()` e `pg_snapshot_xmin()`.

  Alterações de transações ainda abertas ficam para o ciclo seguinte, em vez de serem puladas.
- O aparelho guarda o maior número recebido como novo cursor, **só depois** de gravar a página localmente.

Aplicação no aparelho:

Situação local do registro	O que acontece com a alteração recebida
Não existe ou SINCRONIZADO	Gravada; situação SINCRONIZADO

Situação local do registro	O que acontece com a alteração recebida
PENDENTE	**Não aplicada.** O próximo envio detectará o conflito pela versão.
CONFLITO	Não aplicada
Mesma versão já presente	Ignorada (repetição)

O primeiro acesso em um aparelho é um recebimento com cursor zero, paginado, exibindo o progresso.

7. Resolução de conflito (UC10)

Mantém a regra já aprovada:

1. O aplicativo busca a versão atual do servidor e a exibe ao lado da local.
2. Depois da escolha do produtor, consulta de novo a versão do servidor.
3. Se for mantida a versão local, envia com versão-base igual à versão exibida. O servidor só aceita se ela ainda for a atual; se não for, o conflito é reapresentado.
4. Se for adotada a versão do servidor, ela substitui a local, que passa a SINCRONIZADO.

8. Tratamento de erros

Situação	Comportamento
Sem conexão ou tempo esgotado (evento `TimedOut` do Web ou erro de rede)	Nova tentativa automática, controlada pelo Clock, com intervalos crescentes: 5 s, 15 s, 45 s, 2 min, até no máximo 10 min entre tentativas.
Sessão vencida	Renovar a sessão e repetir. Se a renovação falhar, pedir novo login **sem apagar** pendências.
Acesso negado (regra de acesso por linha)	Registro REJEITADO e erro registrado para diagnóstico. Indica defeito, não uso normal.
Versão do aplicativo incompatível	Interromper a sincronização e pedir atualização do aplicativo. Os dados locais continuam utilizáveis offline.
Erro inesperado do servidor	Tratar como falha de rede (nova tentativa) e registrar para diagnóstico.

9. Compatibilidade de versões

- O servidor mantém a **versão mínima aceita do aplicativo**. Lotes de versões mais antigas são recusados com um código específico.
- Toda mudança de estrutura no servidor deve continuar aceitando a versão mínima vigente. Campos novos são opcionais até que todas as versões em uso os enviem.

10. Registro para diagnóstico

O aplicativo registra localmente, e envia quando houver conexão:

- falhas de sincronização (tipo, código, registro afetado);
- conflitos;
- rejeições.

Nenhum dado pessoal ou financeiro vai nesse registro, só identificadores e códigos.

11. Pontos a confirmar na implementação

- Comportamento transacional exato das chamadas RPC via API do Supabase (PostgREST). A página de documentação correspondente não pôde ser consultada nesta revisão.
- Formato exato da lista de cabeçalhos em `RequestHeaders` do componente Web e tratamento dos códigos de resposta HTTP no evento `GotText`, a confirmar no primeiro teste do protótipo (fase 1).
- Tempo de processamento dos lotes em blocos no aparelho: se 100 registros por lote ficarem lentos, reduzir o tamanho do lote.
- Tamanhos de lote e intervalos (100 / 500 / 5 s): valores iniciais, a ajustar com medições no protótipo.

Especificação 02 — Mapa: fonte de imagem e uso sem conexão

Complementa as seções 1.4 e 7.2 da proposta técnica. Define a fonte da imagem de fundo do componente Map do App Inventor e o comportamento do mapa sem conexão.

1. Requisito que orienta a decisão

O projeto usa apenas os componentes nativos do App Inventor. O componente Map carrega as imagens de fundo pela internet e não oferece, na documentação oficial, download antecipado nem armazenamento de imagens para uso offline. O método `Save` do Map grava apenas os elementos desenhados (em GeoJSON), não as imagens.

Por isso, o que precisa estar sempre disponível no aparelho são os **dados** das unidades. A imagem de satélite é um complemento que aparece quando há conexão.

2. O que funciona com e sem conexão

Elemento	Com conexão	Sem conexão
Contornos (Polygon), vértices (Marker), nomes e áreas	Sim	Sim, a partir do TinyDB
Posição do produtor (ShowUser, LocationSensor)	Sim	Sim, o GPS não depende de internet
Desenho e edição de unidades (TapAtPoint, GPS, digitação)	Sim	Sim
Imagem de satélite de fundo	Sim	Não garantida; fundo neutro

O aplicativo não depende de cache de imagens. Se o componente guardar imagens já vistas, isso é um benefício, não um requisito.

3. Fontes de imagem avaliadas (termos consultados em nov/2026)

Como o uso agora é online, a restrição deixa de ser o armazenamento offline e passa a ser a permissão de uso pela internet num aplicativo.

Fonte	Como usar no App Inventor	Observação	Atende?
OpenStreetMap (padrão do Map, `MapType` = Roads)	Nativo	Mapa de ruas, sem imagem de satélite. A política de uso dos servidores do OpenStreetMap admite uso moderado com atribuição e proíbe download em massa.	Sim, como camada de referência
USGS (`MapType` = Aerial)	Nativo	Imagem aérea do Serviço Geológico dos Estados Unidos. A cobertura e o detalhe no Rio Grande do Sul precisam ser verificados no protótipo.	A verificar
EOX Sentinel-2 cloudless (`MapType` = Custom, `CustomUrl`)	Nativo, por URL de blocos	Gratuito para uso **não comercial** (CC BY-NC-SA 4.0), com atribuição. Resolução de 10 m, cobertura mundial.	Sim para o protótipo
Mapbox, MapTiler, Google, Esri	`CustomUrl`, quando os termos permitem	Exigem conta e chave de acesso, e os termos limitam o uso dos blocos fora dos seus próprios SDKs.	Não nesta fase

4. Decisão

1. **Imagem de fundo nesta fase: mosaico Sentinel-2 cloudless da EOX**, configurado pela propriedade `CustomUrl` do componente Map, no formato de URL de blocos `{z}/{x}/{y}` aceito pelo componente. O endereço exato do serviço de blocos da EOX é confirmado na fase 1 do protótipo. O projeto está em fase acadêmica e sem fins comerciais, o que atende à licença CC BY-NC-SA 4.0.
2. **Alternativa:** se o serviço da EOX não funcionar com o componente, usar `MapType` = Aerial (USGS), desde que a cobertura no Rio Grande do Sul seja adequada, ou Roads (OpenStreetMap), que mostra estradas e rios.

Limitação a declarar: imagens de 10 m mostram talhões, estradas e mudanças de cultura, mas não detalhes finos como cercas ou carreadores estreitos. A precisão do contorno vem do GPS e das coordenadas, não da imagem.

5. Atribuição obrigatória na tela do mapa

Exibir, de forma discreta e acessível, por exemplo num rótulo no canto do mapa ou num ícone “i”:

- “© colaboradores do OpenStreetMap” (quando o fundo for Ruas);
- o texto exigido pela fonte de imagem. Para o EOX cloudless: “EOxCloudless by EOX IT Services GmbH (Contains modified Copernicus Sentinel data [ano])”.

6. Uso comercial (pendente, só se houver)

- Contratar imagem com licença comercial e permissão de uso pelo componente Map, ou servir um mosaico Sentinel-2 processado pelo próprio projeto.
- Mapa de fundo offline exigiria extensão do App Inventor ou outra plataforma de desenvolvimento (trabalho futuro na seção 7.7 da proposta).

Fontes consultadas

- MIT App Inventor, *Maps Components* (Map, Polygon, Marker, FeatureCollection): https://ai2a.appinventor.mit.edu/reference/components/maps.html
- MIT App Inventor, *Sensor Components* (LocationSensor): https://ai2a.appinventor.mit.edu/reference/components/sensors.html
- OSM Foundation, *Tile Usage Policy*: https://operations.osmfoundation.org/policies/tiles/
- EOX, *EOxCloudless license*: https://cloudless.eox.at/documentation/license

Especificação 03 — Ajustes do modelo de dados

Complementa a seção 6 da proposta técnica. Lista as mudanças necessárias no modelo antes da criação das tabelas, para dar suporte à sincronização (especificação 01) e à segurança, considerando o aplicativo feito no MIT App Inventor, com armazenamento local no TinyDB.

1. Mudanças nas tabelas existentes

Tabela	Mudança	Motivo
UNIDADE_MANEJO, VERTICE, ATIVIDADE	Acrescentar `produtor_id` em VERTICE e ATIVIDADE (UNIDADE_MANEJO já tem).	As regras de acesso por linha e as consultas de sincronização filtram direto pelo dono, sem consultar outra tabela.
Todas	Acrescentar `criado_em`, definido pelo servidor.	Auditoria e diagnóstico.
UNIDADE_MANEJO, ATIVIDADE	Manter `versao` e `atualizado_em`, ambos definidos **só** pelo servidor.	Controle de concorrência (especificação 01).
UNIDADE_MANEJO	`area_hectares` passa a ser **recalculada pelo servidor** (PostGIS) a cada gravação aceita.	Define a fonte da verdade da área. O valor do aparelho é provisório até a sincronização.
VERTICE	Sem `versao` própria; os vértices são regravados em bloco com a unidade.	O contorno é um bloco único.
ATIVIDADE	Custos e receita em reais com 2 casas decimais no servidor; **centavos como número inteiro** no aparelho.	Os números dos blocos do App Inventor são de ponto flutuante; somas em ponto flutuante geram diferenças de centavos.
VERTICE	Latitude e longitude com 6 casas decimais.	Precisão de cerca de 0,1 m, suficiente para GPS de celular.

2. Regras de integridade (servidor)

- Valores monetários, tempo estimado e área: **nunca negativos**.
- `tipo` da atividade restrito a: AMOSTRAGEM_MONITORAMENTO, PLANTIO, APLICACAO_DEFENSIVO, ADUBACAO, CORRECAO_SOLO, COLHEITA, MANEJO_PASTAGEM, ROCADA, OUTRA (seção 8).
- `uso` da unidade restrito a: LAVOURA, PASTAGEM, MATA, OUTRO (seção 8).
- `avaliacao`: nula, POSITIVA ou NEGATIVA; não nula somente com `situacao_execucao` = CONCLUIDA.

- `situacao_execucao`: PLANEJADA, EM_ANDAMENTO ou CONCLUIDA; CONCLUIDA exige `data_realizada`.
- `origem` do vértice: MANUAL, GEOLOCALIZACAO ou MAPA.
- Par (`unidade_id`, `ordem`) único.
- Latitude entre −90 e 90; longitude entre −180 e 180.
- Polígono válido e com no mínimo 3 vértices, verificado na função de envio (especificação 01, seção 5).

3. Novas tabelas no servidor

Tabela	Conteúdo	Uso
REGISTRO_ALTERACAO	número sequencial, `produtor_id`, tipo do registro, identificador do registro, nova versão, identificador da transação, data	Cursor de recebimento sem pular registros.
OPERACAO_PROCESSADA	identificador da operação, `produtor_id`, resultado, data	Idempotência dos envios. Pode ser limpa após 30 dias.
CONFIGURACAO_APP	versão mínima aceita do aplicativo	Compatibilidade de versões.

4. Armazenamento no aparelho (TinyDB)

O App Inventor não tem banco relacional nativo. O aparelho usa o componente TinyDB, um armazenamento persistente de pares chave-valor, com um único espaço de nomes (`Namespace` = `agromanejo`). Cada valor é um dicionário ou uma lista, convertidos automaticamente pelo componente.

Chave	Conteúdo	Uso
`propriedade`	dicionário da propriedade	Somente leitura no aparelho.
`unidade:<id>`	dicionário da unidade com a lista ordenada de vértices	Registro de trabalho.
`atividade:<id>`	dicionário da atividade	Registro de trabalho.
`indice:unidades`	lista dos identificadores das unidades	Listar sem percorrer todas as chaves.
`indice:atividades:<unidade_id>`	lista dos identificadores das atividades da unidade	Listar as atividades da unidade.
`indice:pendentes`	lista dos identificadores em PENDENTE	Montar o lote de envio sem percorrer tudo.
`meta:sincronizacao`	cursor de recebimento, data do último ciclo bem-sucedido	Retomar o recebimento de onde parou.

Chave	Conteúdo	Uso
`sessao`	token de acesso, token de renovação, validade, id do produtor	Autenticação nas chamadas do componente Web.
`diagnostico`	lista das últimas 100 falhas: data, tipo de evento, código, id do registro	Diagnóstico de falhas, sem dados pessoais.

Nos dicionários de unidade e atividade, acrescentar:

- `situacao`: SINCRONIZADO, PENDENTE, CONFLITO ou **REJEITADO** (novo estado; ver especificação 01, seção 3);
- `versao_base`;
- `id_operacao_pendente`;
- `motivo_rejeicao`.

Regras de gravação:

- toda gravação de registro atualiza também os índices afetados no mesmo procedimento;
- ao sair da conta, `ClearAll` apaga todas as chaves do espaço de nomes;
- durante o desenvolvimento com o AI Companion, os projetos compartilham o mesmo TinyDB; testes de persistência definitivos usam o APK instalado.

5. Índices

Tabela	Índice	Motivo
UNIDADE_MANEJO, VERTICE, ATIVIDADE	`produtor_id` (primeira coluna do índice)	A documentação do Supabase recomenda indexar toda coluna usada em regras de acesso; sem índice, a leitura vira varredura completa.
VERTICE	(`unidade_id`, `ordem`), único	Contorno ordenado.
ATIVIDADE	`unidade_id`	Listar atividades da unidade.
REGISTRO_ALTERACAO	(`produtor_id`, número sequencial)	Recebimento paginado por produtor.

6. Regras de acesso por linha

Seguindo o padrão recomendado pela documentação do Supabase:

- uma regra por operação (leitura, inclusão, alteração), sempre aplicada ao papel de usuário autenticado;
- condição `produtor_id` igual ao usuário autenticado, com a função de usuário avaliada uma única vez por consulta e verificação explícita de usuário não nulo;

- as gravações do aplicativo passam pela função de envio (especificação 01). As tabelas podem ficar sem permissão de gravação direta para o papel autenticado, o que reduz a superfície de ataque.

7. Impacto nos diagramas da proposta

Ao aprovar estes ajustes, atualizar na proposta:

- **Modelo ER:** `produtor_id` em VERTICE e ATIVIDADE, `criado_em`, e as tabelas de servidor da seção 3;
- **Diagrama de classes:**
  - `SituacaoSincronizacao` passa a ter REJEITADO;
  - uma interface comum `RegistroSincronizavel` (`id`, `versao`, `versaoBase`, `situacao`) implementada por `UnidadeManejo` e `Atividade`;
  - as operações de sincronização passam a usar esse tipo em vez de `Object`;
- **Seção 7.4:** referenciar esta especificação e a 01.
- **Seção 6:** armazenamento local no TinyDB, sem tabela de pacotes de mapa.

8. Decisões funcionais que afetam o modelo (decididas)

Decididas antes da criação das tabelas, porque mudam colunas e a sincronização. Já refletidas na proposta (seções 1.5, 2, 3, 4 e 6 e Figuras 2a, 2b, 5 e 8a).

Decisão	Situação	Mudança no modelo
Exclusão de unidades e atividades	**Aprovada**	Coluna `excluido_em` (exclusão lógica) em UNIDADE_MANEJO e ATIVIDADE, propagada pela sincronização como alteração comum. Quando a exclusão de uma unidade é aceita, a função de envio exclui na mesma transação as atividades ainda ativas da unidade e registra cada uma em REGISTRO_ALTERACAO. Atividade nova ou alterada numa unidade excluída é REJEITADA. Exclusão contra edição feita em outro aparelho é conflito comum (UC10). Sem lixeira nesta versão.

Decisão	Situação	Mudança no modelo
Data e situação da atividade	**Aprovada**	Colunas `data_prevista` (opcional), `situacao_execucao` (PLANEJADA, EM_ANDAMENTO, CONCLUIDA; padrão PLANEJADA) e `data_realizada` (obrigatória quando CONCLUIDA).
Valores realizados	**Aprovada**	Colunas `custo_mao_de_obra_realizado`, `custo_insumos_realizado`, `custo_maquinario_realizado` e `receita_realizada`, opcionais e nunca negativas. O aparelho as pré-preenche com os valores previstos ao concluir. No aparelho, em centavos inteiros, como os previstos.
Uso da unidade e novos tipos de atividade	**Aprovada**	Coluna `uso` em UNIDADE_MANEJO (LAVOURA, PASTAGEM, MATA, OUTRO) e quatro novos valores de `tipo`: APLICACAO_DEFENSIVO, CORRECAO_SOLO, MANEJO_PASTAGEM, ROCADA. Atendem lavoura e pastagem sem novas entidades.
Propriedade	**Aprovada**	Nova tabela PROPRIEDADE (`id`, `produtor_id`, `nome`, `criado_em`) e coluna `propriedade_id` em UNIDADE_MANEJO, com índice. Criada pelo servidor quando a conta é confirmada (uma por produtor nesta versão) e enviada ao aparelho pelo recebimento; o aparelho não a altera. As regras de acesso continuam por `produtor_id`.
Perfis de equipe	**Adiada** (trabalho futuro)	Quando implementada: entidade de membros da propriedade e regras de acesso por propriedade. A tabela PROPRIEDADE já prepara essa mudança.

Decisão	Situação	Mudança no modelo
Rebanho	**Adiada** (trabalho futuro)	Entidades de animais ou lotes, pesagens, sanidade, reprodução e movimentação entre piquetes, ligadas às unidades com uso PASTAGEM.